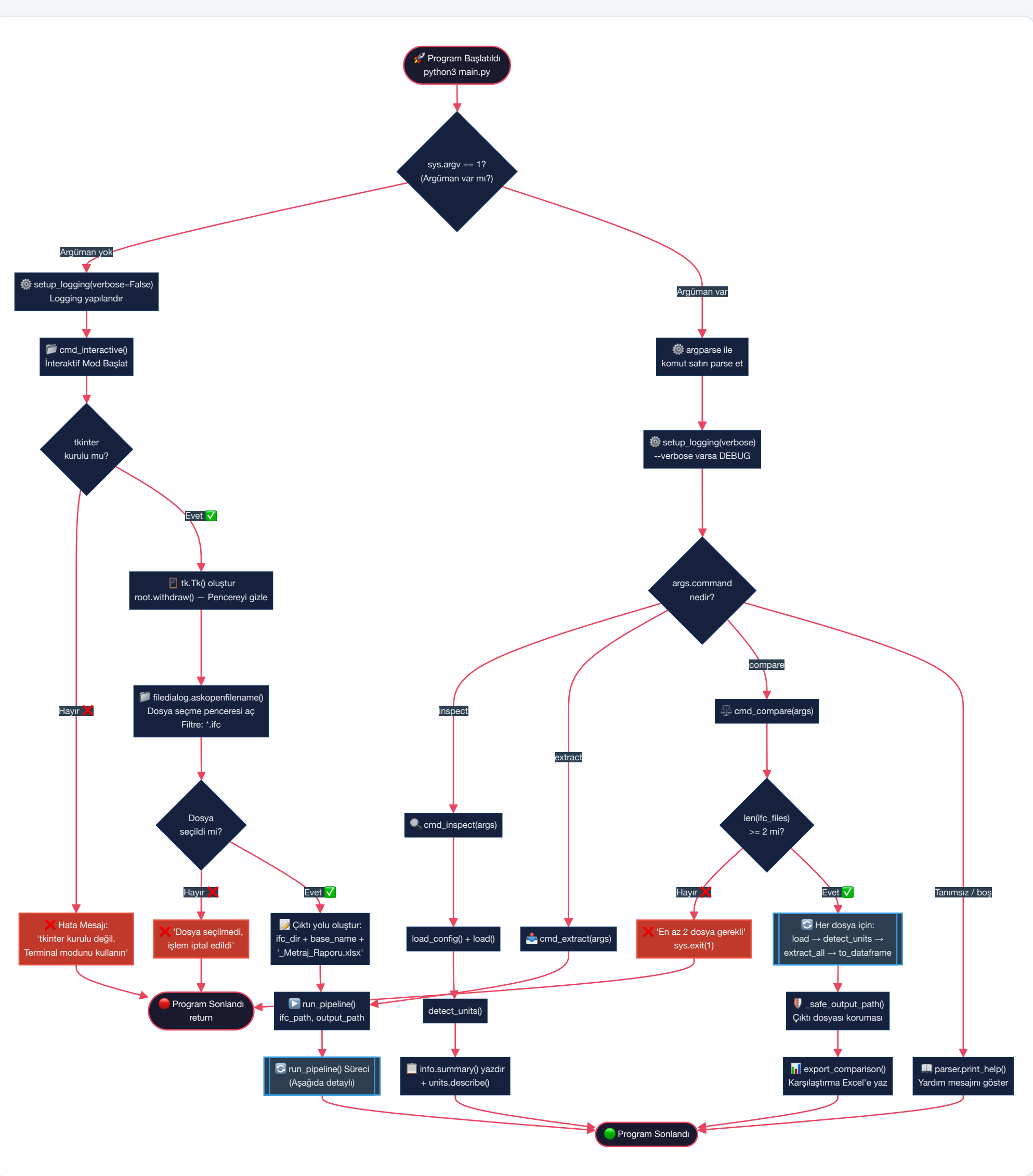
IFC Pipeline — Kapsamlı Algoritma Akış Diyagramı

Yazılımın tüm çalışma senaryolarını, hata giderme ve ayıklama süreçlerini gösteren kutulu akış diyagramları.

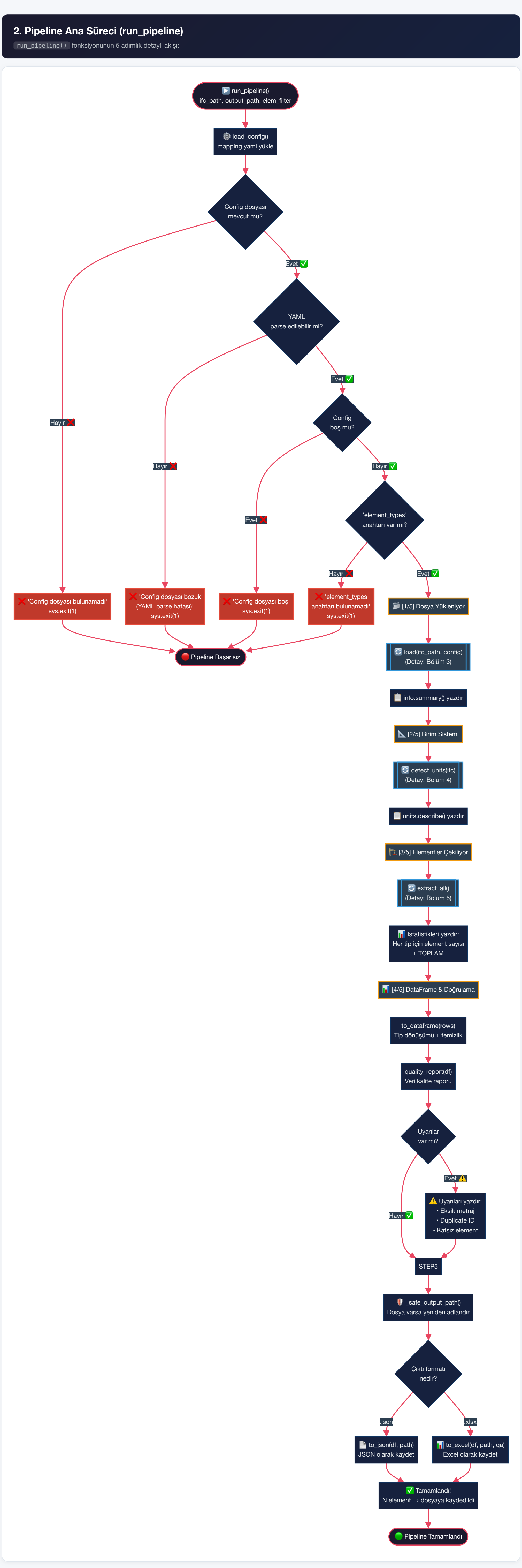
1. Ana Program Akışı (main.py)

Programın başlatılmasından komut yönlendirmesine kadar olan üst düzey akış:


Program Başlatıldı
python3 main.py
sys.argv == 1?
(Argüman var mı?)
Argüman yok
setup_logging(verbose=False)
Logging yapılandır
cmd_interactive()
İnteraktif Mod Başlat
tkinter
kurulu mu?
Evet ✅
tk.Tk() oluştur
root.withdraw() — Pencereyi gizle
filedialog.askopenfilename()
Dosya seçme penceresi aç
Filtre: *.ifc
Dosya
seçildi mi?
Hayır ❌
Evet ✅
Hayır ❌
❌ Hata Mesajı:
'tkinter kurulu değil.
Terminal modunu kullanın'
❌ 'Dosya seçilmedi,
işlem iptal edildi'
Çıktı yolu oluştur:
ifc_dir + base_name +
'_Metraj_Raporu.xlsx'
Program Sonlandı
return
run_pipeline()
ifc_path, output_path
run_pipeline() Süreci
(Aşağıda detaylı)
Argüman var
argparse ile
komut satırı parse et
setup_logging(verbose)
--verbose varsa DEBUG
args.command
nedir?
inspect
extract
compare
Tanımsız / boş
cmd_inspect(args)
load_config() + load()
detect_units()
info.summary() yazdır
+ units.describe()
cmd_extract(args)
cmd_compare(args)
len(ifc_files)
>= 2 mi?
Hayır ❌
Evet ✅
❌ 'En az 2 dosya gerekli'
sys.exit(1)
Her dosya için:
load → detect_units →
extract_all → to_dataframe
_safe_output_path()
Çıktı dosyası koruması
export_comparison()
Karşılaştırma Excel'e yaz
parser.print_help()
Yardım mesajını göster
Program Sonlandı

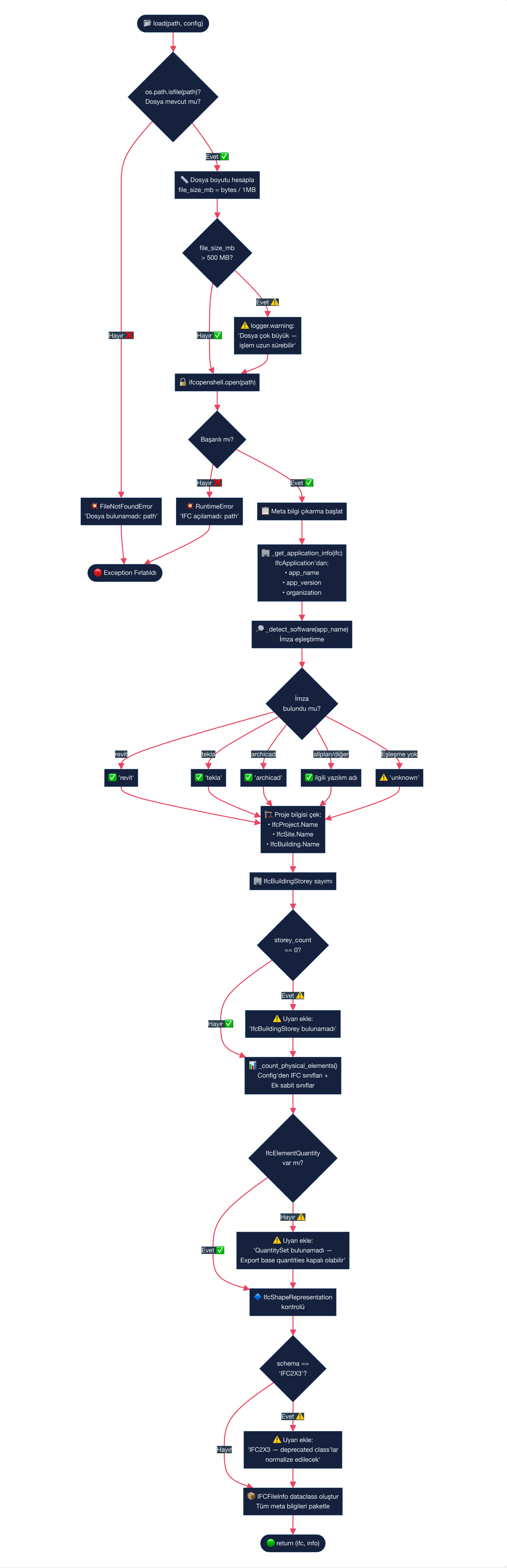
2. Pipeline Ana Süreci (run_pipeline)

`run_pipeline()` fonksiyonunun 5 adımlık detaylı akışı:


run_pipeline()
ifc_path, output_path, elem_filter
load_config()
mapping.yaml yükle
Config dosyası
mevcut mu?
Evet ✅
YAML
parse edilebilir mi?
Evet ✅
Config
boş mu?
Hayır ✅
'element_types'
anahtarı var mı?
Hayır
Hayır
Evet ❌
Hayır ❌
Evet ✅
❌ 'Config dosyası bulunamadı'
sys.exit(1)
❌ 'Config dosyası bozuk
(YAML parse hatası)'
sys.exit(1)
❌ 'Config dosyası boş'
sys.exit(1)
❌ 'element_types
anahtarı bulunamadı'
sys.exit(1)
Pipeline Başarısız
[1/5] Dosya Yükleniyor
load(ifc_path, config)
(Detay: Bölüm 3)
info.summary() yazdır
[2/5] Birim Sistemi
detect_units(ifc)
(Detay: Bölüm 4)
units.describe() yazdır
[3/5] Elementler Çekiliyor
extract_all()
(Detay: Bölüm 5)
İstatistikleri yazdır:
Her tip için element sayısı
+ TOPLAM
[4/5] DataFrame & Doğrulama
to_dataframe(rows)
Tip dönüşümü + temizlik
quality_report(df)
Veri kalite raporu
Uyarılar
var mı?
Evet ⚠️
Hayır ✅
⚠️ Uyarıları yazdır:
• Eksik metraj
• Duplicate ID
• Katsız element
STEP5
_safe_output_path()
Dosya varsa yeniden adlandır
Çıktı formatı
nedir?
.json
.xlsx
to_json(df, path)
JSON olarak kaydet
to_excel(df, path, qa)
Excel olarak kaydet
✅ Tamamlandı!
N element → dosyaya kaydedildi
Pipeline Tamamlandı

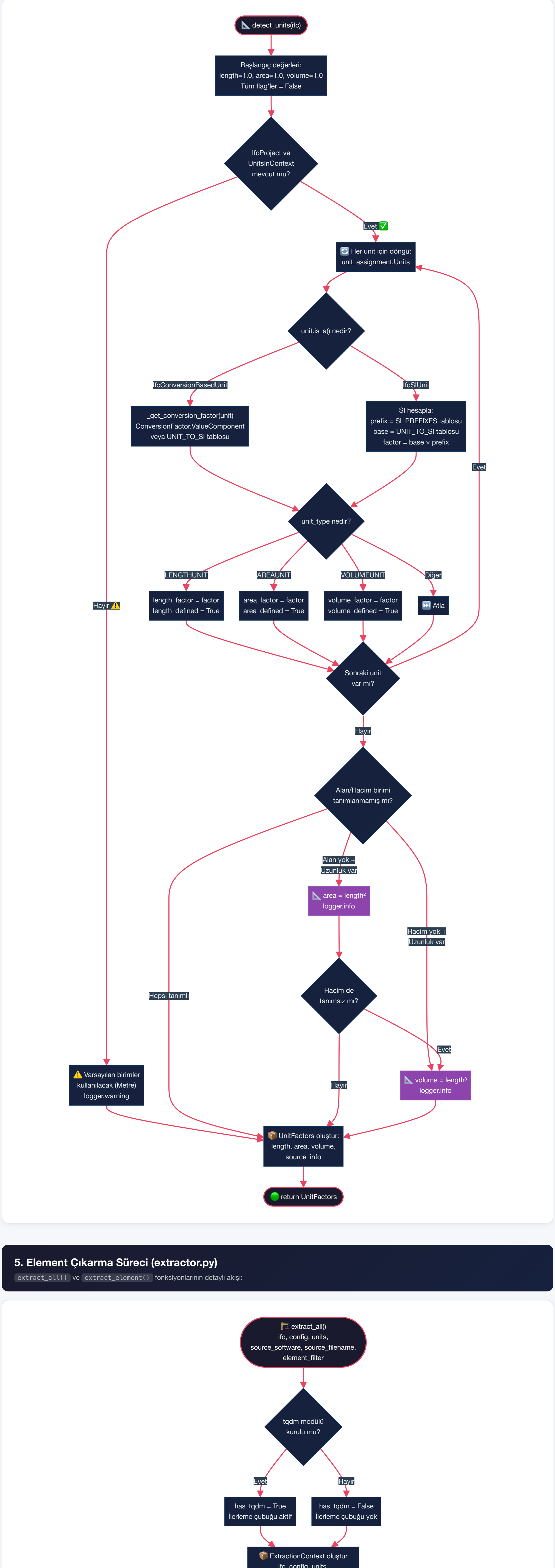
3. IFC Dosya Yükleme Süreci (loader.py)

`load()` fonksiyonunun dosya açma, meta bilgi çıkarma ve yazılım tespiti akışı:


load(path, config)
os.path.isfile(path)?
Dosya mevcut mu?
Evet ✅
Dosya boyutu hesapla
file_size_mb = bytes / 1MB
file_size_mb
> 500 MB?
Evet ⚠️
⚠️ logger.warning:
'Dosya çok büyük —
işlem uzun sürebilir'
Hayır ❌
Hayır ✅
ifcopenshell.open(path)
Başarılı mı?
Hayır ❌
Evet ✅
FileNotFoundError
'Dosya bulunamadı: path'
RuntimeError
'IFC açılamadı: path'
Exception Fırlatıldı
Meta bilgi çıkarma başlat
_get_application_info(ifc)
IfcApplication'dan:
• app_name
• app_version
• organization
_detect_software(app_name)
İmza eşleştirme
İmza
bulundu mu?
revit
tekla
archicad
allplan/diğer
Eşleşme yok
✅ 'revit'
✅ 'tekla'
✅ 'archicad'
✅ ilgili yazılım adı
⚠️ 'unknown'
Proje bilgisi çek:
• IfcProject.Name
• IfcSite.Name
• IfcBuilding.Name
IfcBuildingStorey sayımı
storey_count
== 0?
Evet ⚠️
Hayır ✅
⚠️ Uyarı ekle:
'IfcBuildingStorey bulunamadı'
_count_physical_elements()
Config'den IFC sınıfları +
Ek sabit sınıflar
IfcElementQuantity
var mı?
Hayır ⚠️
Evet ✅
⚠️ Uyarı ekle:
'QuantitySet bulunamadı —
Export base quantities kapalı olabilir'
IfcShapeRepresentation
kontrolü
schema ==
'IFC2X3'?
Evet ⚠️
Hayır
⚠️ Uyarı ekle:
'IFC2X3 — deprecated class'lar
normalize edilecek'
IFCFileInfo dataclass oluştur
Tüm meta bilgileri paketle
return (ifc, info)

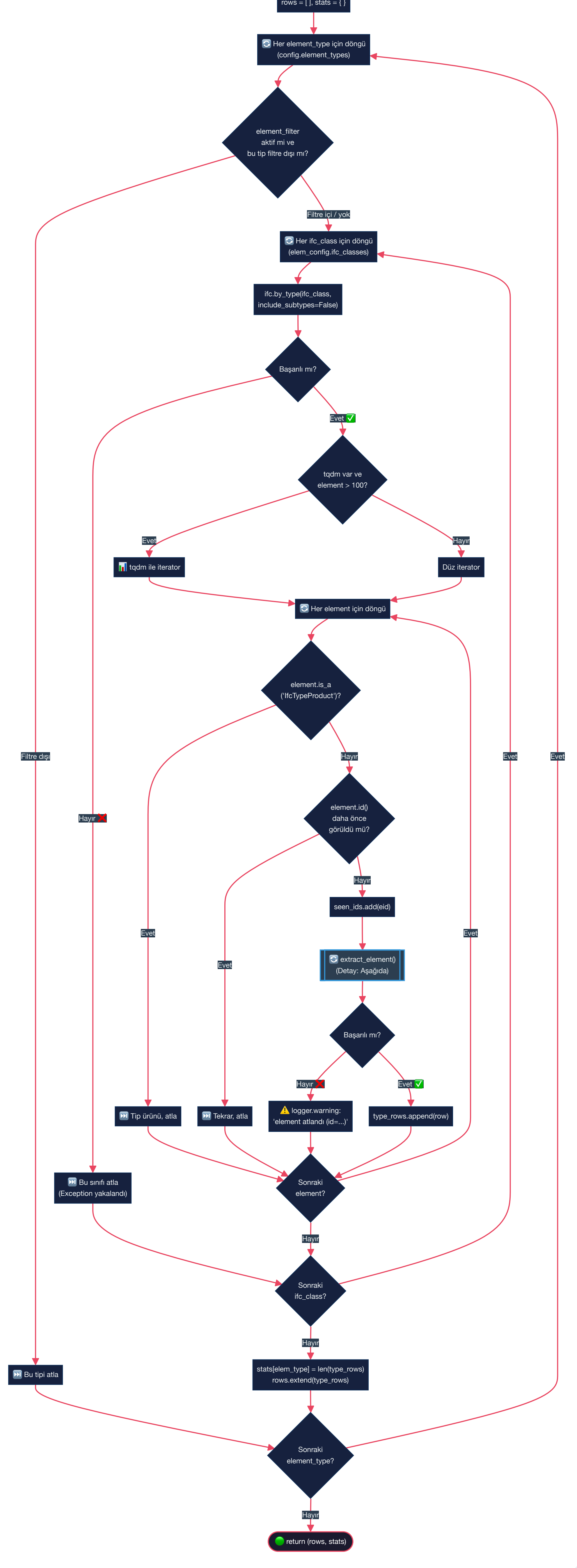
4. Birim Tespiti Süreci (units.py)

`detect_units()` fonksiyonunun uzunluk, alan ve hacim birimlerini bağımsız tespit etme akışı:


detect_units(ifc)
Başlangıç değerleri:
length=1.0, area=1.0, volume=1.0
Tüm flag'ler = False
IfcProject ve
UnitsInContext
mevcut mu?
Evet ✅
Her unit için döngü:
unit_assignment.Units
unit.is_a() nedir?
IfcConversionBasedUnit
IfcSIUnit
_get_conversion_factor(unit)
ConversionFactor.ValueComponent
veya UNIT_TO_SI tablosu
SI hesapla:
prefix = SI_PREFIXES tablosu
base = UNIT_TO_SI tablosu
factor = base × prefix
Evet
unit_type nedir?
LENGTHUNIT
AREAUNIT
VOLUMEUNIT
Diğer
length_factor = factor
length_defined = True
area_factor = factor
area_defined = True
volume_factor = factor
volume_defined = True
Atla
Hayır ⚠️
Sonraki unit
var mı?
Hayır
Alan/Hacim birimi
tanımlanmamış mı?
Alan yok +
Uzunluk var
area = length²
logger.info
Hacim yok +
Uzunluk var
Hepsi tanımlı
Hacim de
tanımsız mı?
Evet
Hayır
volume = length³
logger.info
⚠️ Varsayılan birimler
kullanılacak (Metre)
logger.warning
UnitFactors oluştur:
length, area, volume,
source_info
return UnitFactors

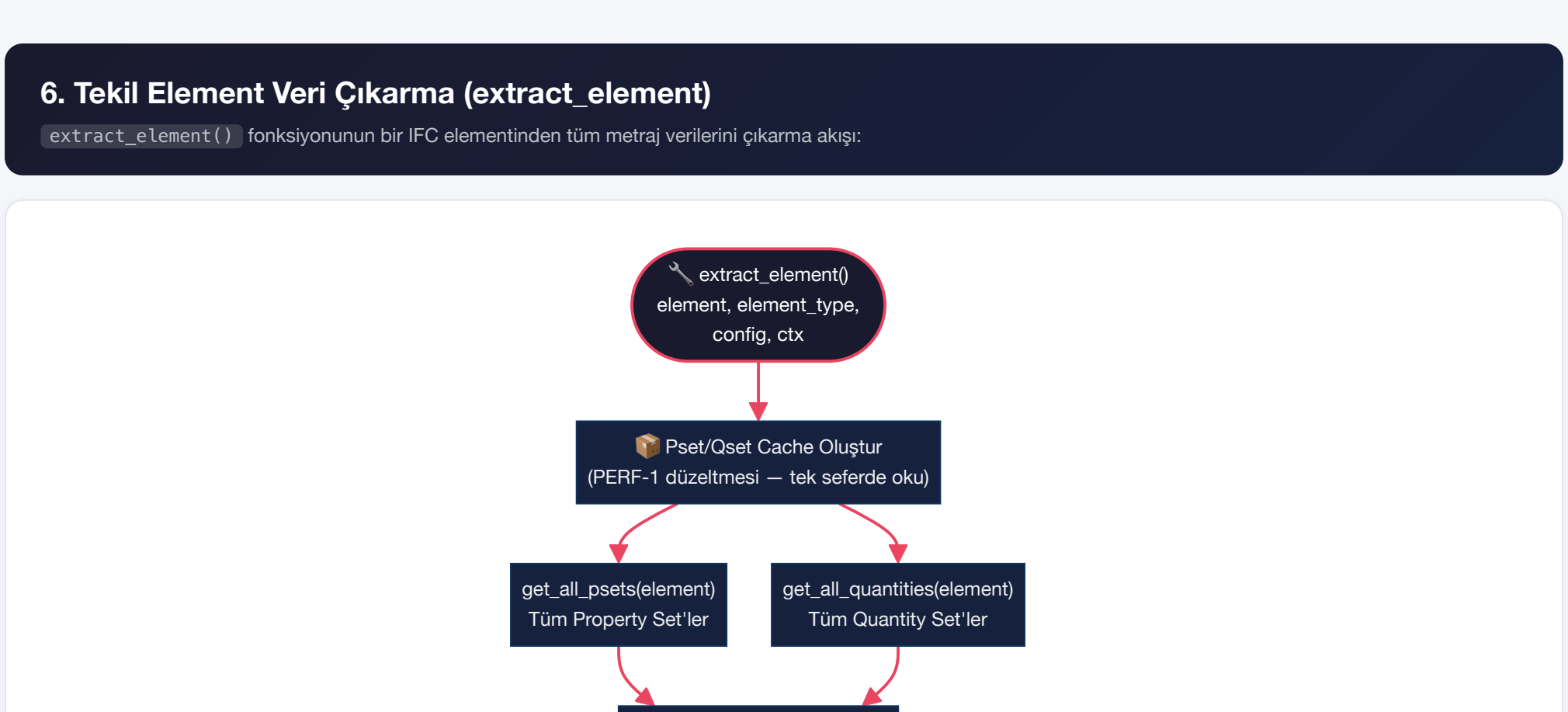
5. Element Çıkarma Süreci (extractor.py)

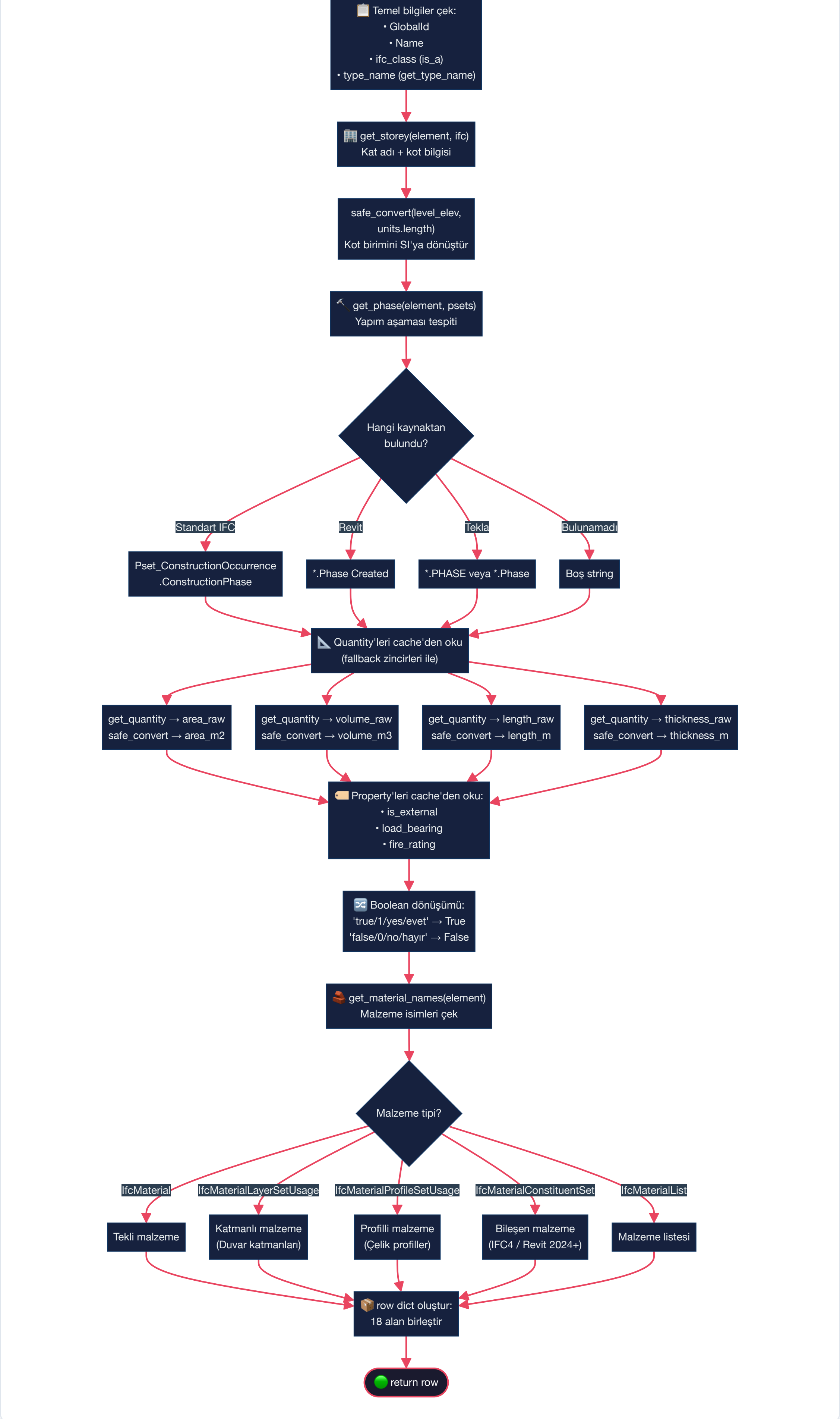
`extract_all()` ve `extract_element()` fonksiyonlarının detaylı akışı:


extract_all()
ifc, config, units,
source_software, source_filename,
element_filter
tqdm modülü
kurulu mu?
Evet
Hayır
has_tqdm = True
İlerleme çubuğu aktif
has_tqdm = False
İlerleme çubuğu yok
ExtractionContext oluştur
ifc, config, units,
source_software, source_filename
rows = [ ], stats = { }
Her element_type için döngü
(config.element_types)
element_filter
aktif mi ve
bu tip filtre dışı mı?
Filtre içi / yok
Filtre dışı
Her ifc_class için döngü
(elem_config.ifc_classes)
ifc.by_type(ifc_class,
include_subtypes=False)
Başarılı mı?
Evet ✅
Hayır ❌
tqdm var ve
element > 100?
Evet
Hayır
tqdm ile iterator
Düz iterator
Her element için döngü
element.is_a
('IfcTypeProduct')?
Hayır
Evet
element.id()
daha önce
görüldü mü?
Hayır
Evet
seen_ids.add(eid)
extract_element()
(Detay: Aşağıda)
Başarılı mı?
Hayır ❌
Evet ✅
Evet
Evet
Evet
Tip ürünü, atla
Tekrar, atla
⚠️ logger.warning:
'element atlandı (id=...)'
type_rows.append(row)
Bu sınıfı atla
(Exception yakalandı)
Sonraki
element?
Hayır
Sonraki
ifc_class?
Hayır
Bu tipi atla
stats[elem_type] = len(type_rows)
rows.extend(type_rows)
Sonraki
element_type?
Hayır
return (rows, stats)

6. Tekil Element Veri Çıkarma (extract_element)

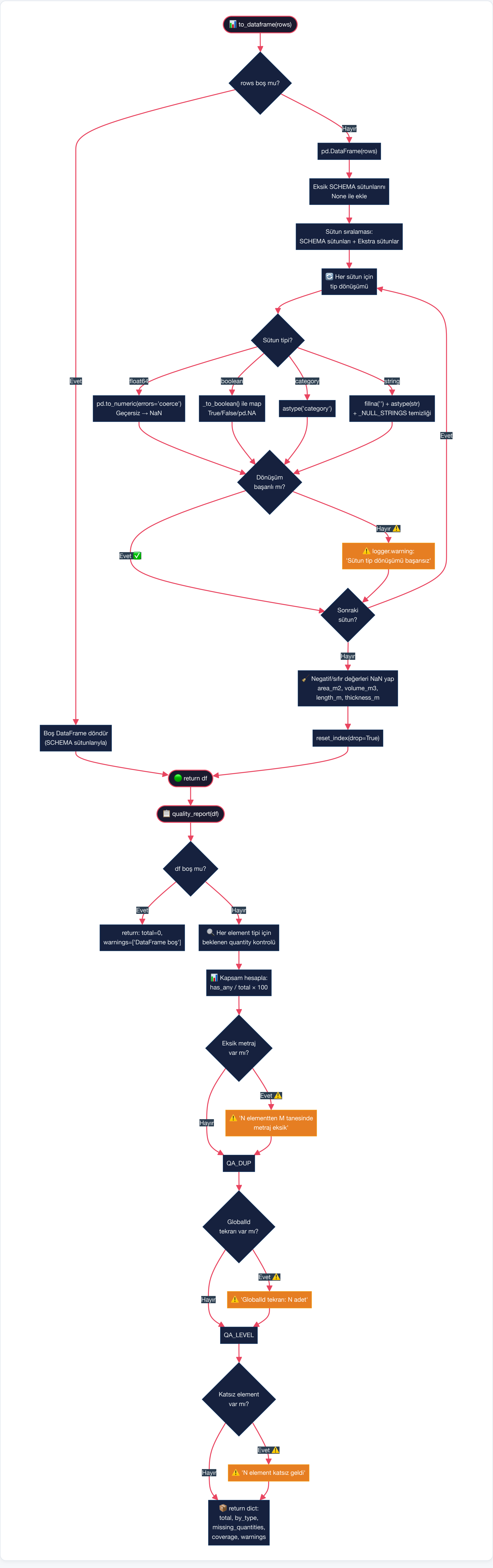
`extract_element()` fonksiyonunun bir IFC elementinden tüm metraj verilerini çıkarma akışı:


extract_element()
element, element_type,
config, ctx
Pset/Qset Cache Oluştur
(PERF-1 düzeltmesi — tek seferde oku)
get_all_psets(element)
Tüm Property Set'ler
get_all_quantities(element)
Tüm Quantity Set'ler


Temel bilgiler çek:
• GlobalId
• Name
• ifc_class (is_a)
• type_name (get_type_name)
get_storey(element, ifc)
Kat adı + kot bilgisi
safe_convert(level_elev, units.length)
Kot birimini SI'ya dönüştür
get_phase(element, psets)
Yapım aşaması tespiti
Hangi kaynaktan bulundu?
Standart IFC
Revit
Tekla
Bulunamadı
Pset_ConstructionOccurrence .ConstructionPhase
*.Phase Created
*.PHASE veya *.Phase
Boş string
Quantity'leri cache'den oku (fallback zincirleri ile)
get_quantity → area_raw safe_convert → area_m2
get_quantity → volume_raw safe_convert → volume_m3
get_quantity → length_raw safe_convert → length_m
get_quantity → thickness_raw safe_convert → thickness_m
Property'leri cache'den oku:
• is_external
• load_bearing
• fire_rating
Boolean dönüşümü:
'true/1/yes/evet' → True
'false/0/no/hayır' → False
get_material_names(element)
Malzeme isimleri çek
Malzeme tipi?
IfcMaterial
IfcMaterialLayerSetUsage
IfcMaterialProfileSetUsage
IfcMaterialConstituentSet
IfcMaterialList
Tekli malzeme
Katmanlı malzeme (Duvar katmanları)
Profilli malzeme (Çelik profiller)
Bileşen malzeme (IFC4 / Revit 2024+)
Malzeme listesi
row dict oluştur: 18 alan birleştir
return row

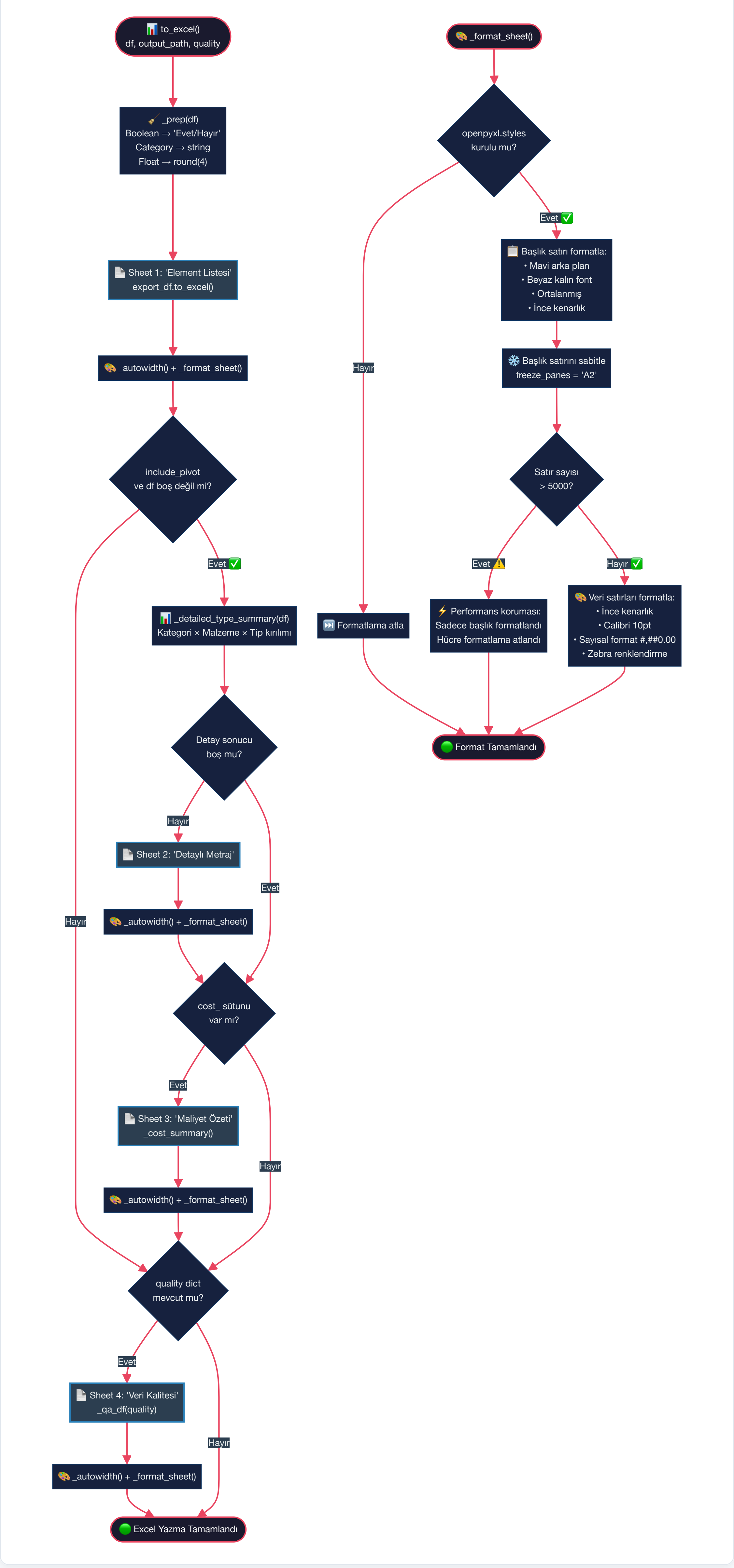
7. Veri Normalizasyon ve Kalite Kontrolü (normalizer.py)

`to_dataframe()` ve `quality_report()` fonksiyonlarının akışı:


to_dataframe(rows)
rows boş mu?
Hayır
pd.DataFrame(rows)
Eksik SCHEMA sütunlarını None ile ekle
Sütun sıralaması: SCHEMA sütunları + Ekstra sütunlar
Her sütun için tip dönüşümü
Sütun tipi?
float64
boolean
category
string
pd.to_numeric(errors='coerce') Geçersiz → NaN
_to_boolean() ile map True/False/pd.NA
astype('category')
fillna('') + astype(str) + _NULL_STRINGS temizliği
Dönüşüm başarılı mı?
Evet ✅
Hayır ⚠️
⚠️ logger.warning: 'Sütun tip dönüşümü başarısız'
Sonraki sütun?
Evet
Hayır
Negatif/sıfır değerleri NaN yap area_m2, volume_m3, length_m, thickness_m
reset_index(drop=True)
Evet
Boş DataFrame döndür (SCHEMA sütunlarıyla)
return df
quality_report(df)
df boş mu?
Evet
Hayır
return: total=0, warnings=['DataFrame boş']
Her element tipi için beklenen quantity kontrolü
Kapsam hesapla: has_any / total × 100
Eksik metraj var mı?
Evet ⚠️
Hayır
⚠️ 'N elementten M tanesinde metraj eksik'
QA_DUP
GlobalId tekrarı var mı?
Evet ⚠️
Hayır
⚠️ 'GlobalId tekrarı: N adet'
QA_LEVEL
Katsız element var mı?
Evet ⚠️
Hayır
⚠️ 'N element katsız geldi'
return dict: total, by_type, missing_quantities, coverage, warnings

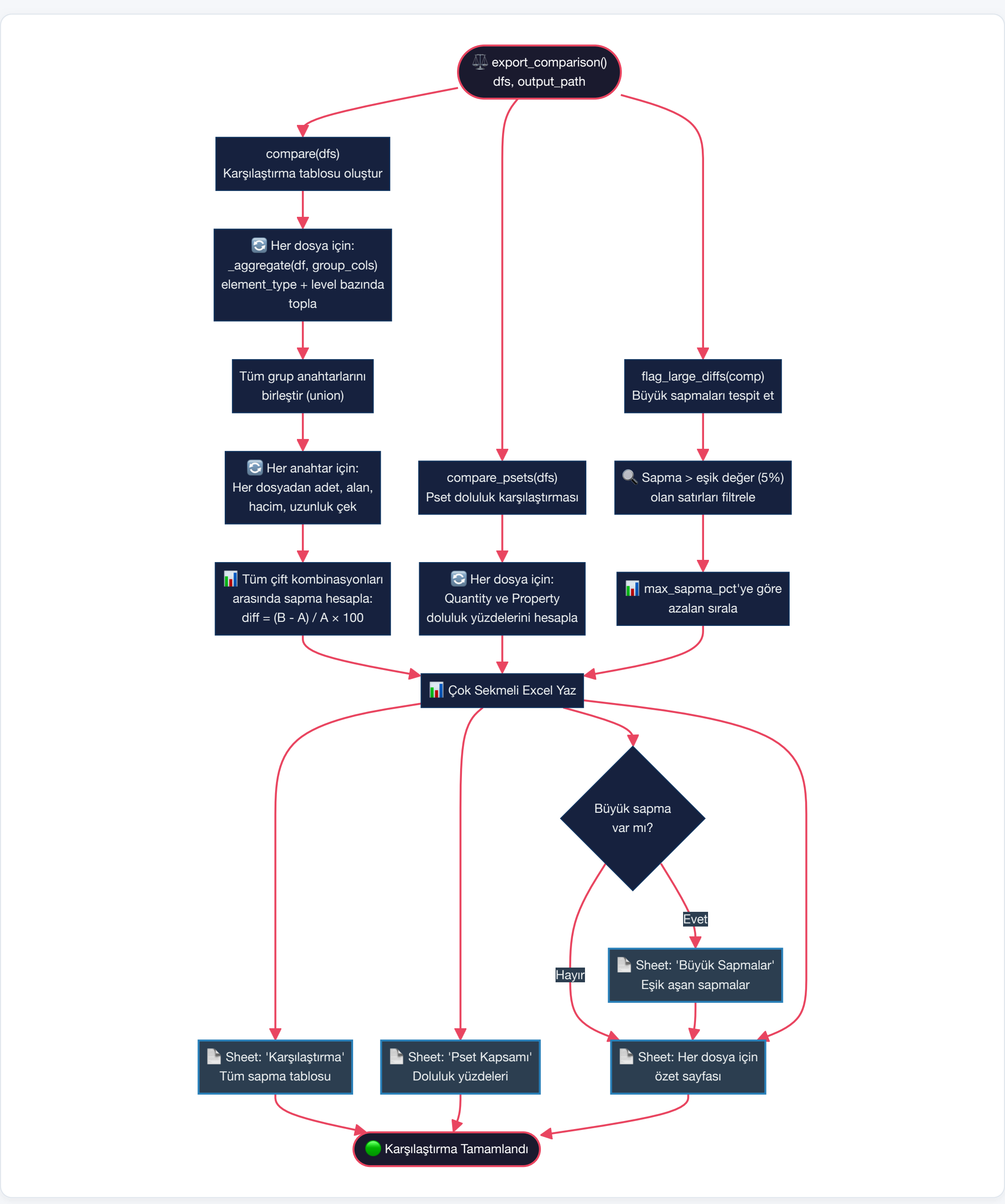
8. Excel Export Süreci (exporter.py)

`to_excel()` fonksiyonunun çok sekmeli, formatlanmış rapor oluşturma akışı:


to_excel()
df, output_path, quality
_prep(df)
Boolean → 'Evet/Hayır'
Category → string
Float → round(4)
Sheet 1: 'Element Listesi'
export_df.to_excel()
_autowidth() + _format_sheet()
include_pivot ve df boş değil mi?
Evet ✅
Hayır
_detailed_type_summary(df)
Kategori × Malzeme × Tip kırılımı
Detay sonucu boş mu?
Hayır
Evet
Sheet 2: 'Detaylı Metraj'
_autowidth() + _format_sheet()
cost_ sütunu var mı?
Evet
Hayır
Sheet 3: 'Maliyet Özeti'
_cost_summary()
_autowidth() + _format_sheet()
quality dict mevcut mu?
Evet
Hayır
Sheet 4: 'Veri Kalitesi'
_qa_df(quality)
_autowidth() + _format_sheet()
Excel Yazma Tamamlandı
_format_sheet()
openpyxl.styles kurulu mu?
Evet ✅
Hayır
Başlık satırı formatla:
• Mavi arka plan
• Beyaz kalın font
• Ortalanmış
• İnce kenarlık
Başlık satırını sabitle freeze_panes = 'A2'
Satır sayısı > 5000?
Evet ⚠️
Hayır ✅
Performans koruması: Sadece başlık formatlandı Hücre formatlama atlandı
Veri satırları formatla:
• İnce kenarlık
• Calibri 10pt
• Sayısal format #,##0.00
• Zebra renklendirme
Formatlama atla
Format Tamamlandı

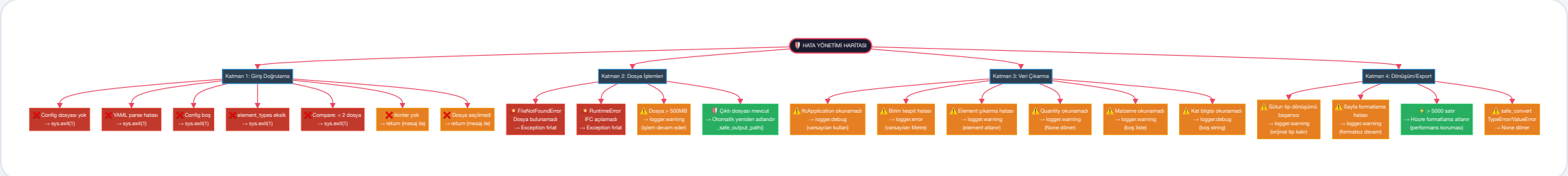
9. Karşılaştırma Süreci (comparator.py)

`export_comparison()` ve ilişkili fonksiyonların akışı:


export_comparison()
dfs, output_path
compare(dfs)
Karşılaştırma tablosu oluştur
Her dosya için:
_aggregate(df, group_cols)
element_type + level bazında topla
Tüm grup anahtarlarını birleştir (union)
Her anahtar için: Her dosyadan adet, alan, hacim, uzunluk çek
Tüm çift kombinasyonları arasında sapma hesapla: diff = (B - A) / A × 100
compare_psets(dfs)
Pset doluluk karşılaştırması
Her dosya için: Quantity ve Property doluluk yüzdelerini hesapla
flag_large_diffs(comp)
Büyük sapmaları tespit et
Sapma > eşik değer (5%) olan satırları filtrele
max_sapma_pct'ye göre azalan sırala
Çok Sekmeli Excel Yaz
Büyük sapma var mı?
Evet
Hayır
Sheet: 'Büyük Sapmalar'
Eşik aşan sapmalar
Sheet: 'Karşılaştırma'
Tüm sapma tablosu
Sheet: 'Pset Kapsamı'
Doluluk yüzdeleri
Sheet: Her dosya için özet sayfası
Karşılaştırma Tamamlandı

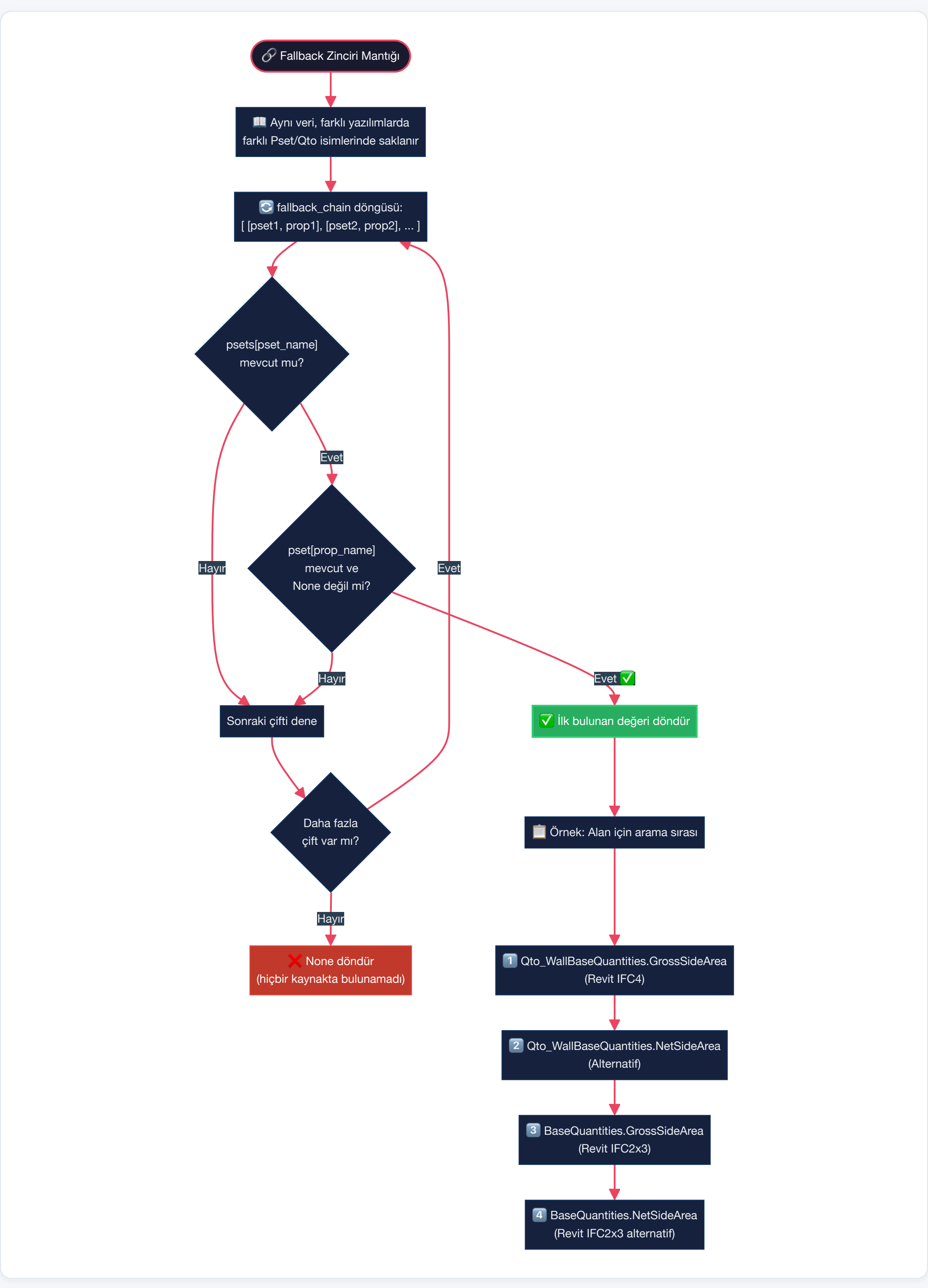
10. Hata Yönetimi Genel Haritası

Yazılımdaki tüm hata yakalama ve geri dönüş noktalarının özeti:


HATA YÖNETİMİ HARİTASI
Katman 1: Giriş Doğrulama
Katman 2: Dosya İşlemleri
Katman 3: Veri Çıkarma
Katman 4: Dönüşüm/Export
Config dosyası yok → sys.exit(1)
YAML parse hatası → sys.exit(1)
Config boş → sys.exit(1)
element_types eksik → sys.exit(1)
Compare: < 2 dosya → sys.exit(1)
tkinter yok → return (mesaj ile)
Dosya seçilmedi → return (mesaj ile)
FileNotFoundError Dosya bulunamadı → Exception fırlat
RuntimeError IFC açılamadı → Exception fırlat
Dosya > 500MB → logger.warning (işlem devam eder)
Çıktı dosyası mevcut → Otomatik yeniden adlandır _safe_output_path()
IfcApplication okunamadı → logger.debug (varsayılan kullan)
Birim tespit hatası → logger.error (varsayılan Metre)
Element çıkarma hatası → logger.warning (element atlanır)
Quantity okunamadı → logger.warning (None döner)
Malzeme okunamadı → logger.warning (boş liste)
Kat bilgisi okunamadı → logger.debug (boş string)
Sütun tip dönüşümü başarısız → logger.warning (orijinal tip kalır)
Sayfa formatlama hatası → logger.warning (formatsız devam)
> 5000 satır → Hücre formatlama atlanır (performans koruması)
safe_convert TypeError/ValueError → None döner

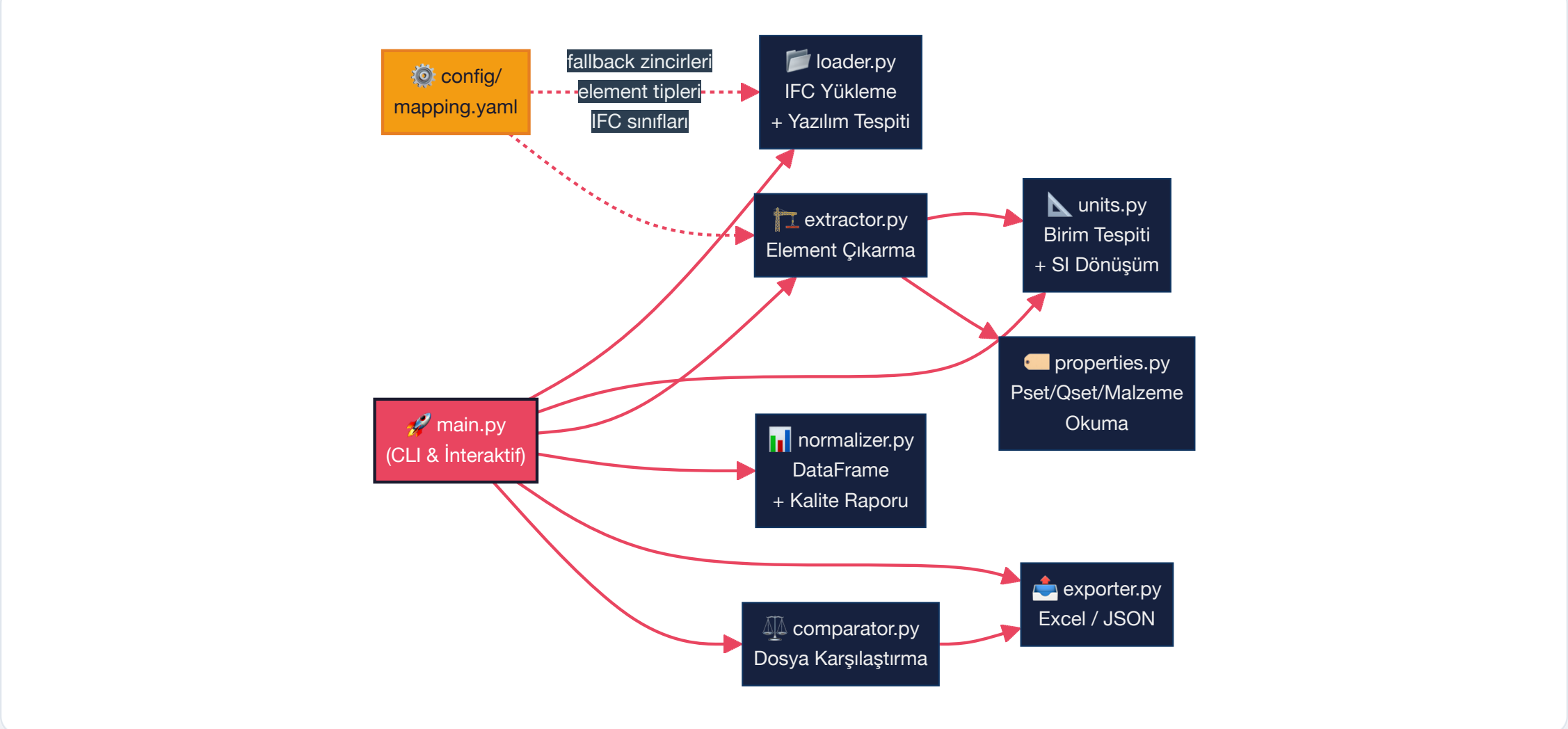
11. Property/Quantity Fallback Zinciri (properties.py)

Farklı yazılımlar için veri arama stratejisi:


Fallback Zinciri Mantığı
Aynı veri, farklı yazılımlarda farklı Pset/Qto isimlerinde saklanır
fallback_chain döngüsü: [[pset1, prop1], [pset2, prop2], ...]
psets[pset_name] mevcut mu?
Evet
Hayır
pset[prop_name] mevcut ve None değil mi?
Evet ✅
Hayır
Sonraki çifti dene
Daha fazla çift var mı?
Evet
Hayır
None döndür (hiçbir kaynakta bulunamadı)
İlk bulunan değeri döndür
Örnek: Alan için arama sırası
1 Qto_WallBaseQuantities.GrossSideArea (Revit IFC4)
2 Qto_WallBaseQuantities.NetSideArea (Alternatif)
3 BaseQuantities.GrossSideArea (Revit IFC2x3)
4 BaseQuantities.NetSideArea (Revit IFC2x3 alternatif)

Modül Bağımlılık Haritası

Yazılımdaki modüller arası veri akışı:


config/ mapping.yaml
fallback zincirleri
element tipleri
IFC sınıfları
loader.py IFC Yükleme + Yazılım Tespiti
extractor.py Element Çıkarma
units.py Birim Tespiti + SI Dönüşüm
properties.py Pset/Qset/Malzeme Okuma
main.py (CLI & İnteraktif)
normalizer.py DataFrame + Kalite Raporu
exporter.py Excel / JSON
comparator.py Dosya Karşılaştırma

IFC Pipeline v0.2.0 — Algoritma Akış Diyagramı
Bu belge otomatik olarak oluşturulmuştur.

🖨️ PDF Olarak Yazdır